SelVerifier: Self-Verifying Programming Language Verification Engine Document

Yusuf Demir
Yağız Kaan Aydoğdu

May 31, 2026

1 Introduction

`SelVeri`, as its name suggests, is a programming language that natively supports verification of first-order logical formulae (including past and future temporal operators) based on the program state and scope. Its subcomponent which is responsible for verifying specifications is called the *verification engine*, or more stylistically, the `SelVerifier`. This documentation focuses on how the verifying process works and its limitations.

2 Syntax of allowed specifications

Let $\texttt{Spec}_{\texttt{sv}}$ be the infinite set of all allowed specifications in `SelVeri`.

$$
\begin{aligned}
\langle Spec \rangle &::= \langle SpecImp \rangle \\
\langle SpecImp \rangle &::= \langle SpecOr \rangle \texttt{ => } \langle SpecImp \rangle \mid \langle SpecOr \rangle \\
\langle SpecOr \rangle &::= \langle SpecOr \rangle \texttt{ || } \langle SpecAnd \rangle \mid \langle SpecAnd \rangle \\
\langle SpecAnd \rangle &::= \langle SpecAnd \rangle \texttt{ \&\& } \langle SpecTemp \rangle \mid \langle SpecTemp \rangle \\
\langle SpecTemp \rangle &::= \langle SpecTemp \rangle \texttt{ Since } \langle SpecUnary \rangle \mid \langle SpecTemp \rangle \texttt{ Until } \langle SpecUnary \rangle \mid \langle SpecUnary \rangle \\
\langle SpecUnary \rangle &::= \texttt{!} \langle SpecUnary \rangle \mid \texttt{Previously } \langle SpecUnary \rangle \mid \texttt{Once } \langle SpecUnary \rangle \mid \texttt{Historically } \langle SpecU \\
&\quad \mid \texttt{Next } \langle SpecUnary \rangle \mid \texttt{Eventually } \langle SpecUnary \rangle \mid \texttt{Always } \langle SpecUnary \rangle \\
&\quad \mid \texttt{Forall } \langle Ident \rangle \texttt{ : } \langle Domain \rangle \texttt{ . } \langle Spec \rangle \mid \texttt{Exists } \langle Ident \rangle \texttt{ : } \langle Domain \rangle \texttt{ . } \langle Spec \rangle \\
&\quad \mid \langle BExp \rangle \mid (\langle Spec \rangle) \\
\langle Domain \rangle &::= [\langle IntLit \rangle \texttt{ ... } \langle IntLit \rangle) \mid [\langle RealLit \rangle \texttt{ , } \langle RealLit \rangle] \mid (\langle RealLit \rangle \texttt{ , } \langle RealLit \rangle) \\
&\quad [\langle RealLit \rangle \texttt{ , } \langle RealLit \rangle) \mid (\langle RealLit \rangle \texttt{ , } \langle RealLit \rangle] \\
&\quad \mid \langle Ident \rangle \mid \langle BasicType \rangle \mid \texttt{Var[} \langle BasicType \rangle \texttt{] } \mid \langle ListLit \rangle
\end{aligned}
$$

3 Verification of First-order Logic formulae

To verify FOL formulae, `SelVerifier` utilizes `Z3 Theorem Prover` (with its `Python API`). In order to map `SelVerIR` program state and scopes and convert specifications from $\texttt{Spec}_{\texttt{SV}}$ to equivalent `Z3` inputs (elements of $\texttt{Spec}_{\texttt{Z3}}$), it utilizes the `Z3Mapper` class, formalized as in the following way:

3.1 Variable Mapping Dictionaries

To accurately preserve the semantics of both the program's memory and the logical specifications, the `Z3Mapper` utilizes two distinct dictionaries for variable resolution:

- `free_var_map`**:** This dictionary maps lexically scoped `SelVeri` program variables to their corresponding Z3 symbolic constants (e.g., `Int`, `Real`, or `Array`). These represent the persistent program state and are globally constrained by the values defined in the runtime environment.
- `bound_var_map`**:** This dictionary tracks temporary variables that are strictly bound by First-Order Logic (FOL) quantifiers (such as $\forall$ and $\exists$) or finite domain iterators within a specification. These map to Z3 bound variables (internally prefixed with `&`) or concrete values during domain unrolling, ensuring they do not interfere with the free variables of the program state.

3.2 Configuration Mapping

The runtime configuration consists of a hierarchical **Scope** (lexical environments) and a **State** (memory values). The mapper enforces a strict lexical depth matching mechanism to preserve vertical isolation.

3.2.1 Lexical Scoping and Variable Naming

To ensure independent history tracking for horizontally isolated blocks, every variable v mapped to `Z3` is prefixed with its owner scope's unique identifier (`scope_id`). We define the naming function $\mathcal{ZN}$ as:

$$\mathcal{ZN}(v, id) = \texttt{\_s}\langle id \rangle \texttt{\_}v$$

3.2.2 Scope Mapping

Let $\mathcal{ZS}$ be the mapping function from `SelVeri` types to `Z3` Sorts. Variables are declared in the `free_var_map` according to their type:

$$\begin{array}{ll} \mathcal{ZS}(\texttt{INT}) & = \texttt{IntSort()} \\ \mathcal{ZS}(\texttt{REAL}) & = \texttt{RealSort()} \\ \mathcal{ZS}(\texttt{LIST[INT]}) & = \texttt{Array}(\texttt{IntSort()}, \texttt{IntSort()}) \\ \mathcal{ZS}(\texttt{LIST[REAL]}) & = \texttt{Array}(\texttt{IntSort()}, \texttt{RealSort()}) \end{array}$$

3.2.3 State Mapping

For a given state mapping variable v to value val, assertions are added to the `Z3` solver to constrain the free variables. If v is of a basic type (`INT` or `REAL`):

$$\texttt{solver.add}(\mathcal{ZN}(v, id) == val)$$

If v is a `LIST` of size n, the array is constrained element by element:

$$\forall i \in [0, n-1] \cdot \texttt{solver.add}(\mathcal{ZN}(v, id)[i] == val[i])$$

3.3 Specification Mapping

The mapper recursively translates Abstract Syntax Tree (AST) nodes of the specification into `Z3` expressions. We define $\mathcal{ZA}$ for arithmetic expressions, $\mathcal{ZB}$ for boolean expressions, and $\mathcal{ZF}$ for generic FOL formulas.

3.3.1 Mapping of Arithmetic Expressions ($\mathcal{ZA}$)

$\mathcal{ZA}(\texttt{IntLit}(v))$	$= \texttt{IntVal}(v)$
$\mathcal{ZA}(\texttt{RealLit}(v))$	$= \texttt{RealVal}(v)$
$\mathcal{ZA}(\texttt{ListLit}(v_0, \ldots, v_n))$	$= \begin{cases} \texttt{Store}(\ldots\texttt{Store}(\texttt{K}(\texttt{Int}, 0), 0, v_0)\ldots, n, v_n) & \text{if all } v_i \in \mathbb{Z} \\ \texttt{Store}(\ldots\texttt{Store}(\texttt{K}(\texttt{Int}, 0.0), 0, v_0)\ldots, n, v_n) & \text{if any } v_i \in \mathbb{R} \end{cases}$
$\mathcal{ZA}(\texttt{AVar}(x))$	$= \texttt{free\_var\_map}[\mathcal{ZN}(x, \text{lexical\_owner}(x))]$
$\mathcal{ZA}(\texttt{ABoundVar}(x))$	$= \texttt{bound\_var\_map}[x]$
$\mathcal{ZA}(\texttt{ALen}(lst))$	$= \begin{cases} \texttt{free\_var\_map}[\mathcal{ZN}(\_lst\_\texttt{len\_1}, id)] & \text{if } lst \text{ is a LIST} \\ \texttt{IntVal}(0) & \text{otherwise} \end{cases}$
$\mathcal{ZA}(\texttt{AIndex}(lst, i_1 \ldots i_d))$	$= \texttt{free\_var\_map}[\mathcal{ZN}(lst, id)][\mathcal{ZA}(LF_{Z3}(lst; i_1, \ldots, i_d))]$
$\mathcal{ZA}(-a)$	$= -\mathcal{ZA}(a)$
$\mathcal{ZA}(a_1 \oplus a_2)$	$= \mathcal{ZA}(a_1) \oplus \mathcal{ZA}(a_2) \quad \text{for } \oplus \in \{+, -, *, /\}$

Note: LF_{Z3} is the `Z3` equivalent of the list flattening logic, computed dynamically using dimension length shadows (_lst_`len`_d) from the mapping scope.

3.3.2 Mapping of Boolean Expressions ($\mathcal{ZB}$)

$$
\begin{array}{ll}
\mathcal{ZB}(\texttt{BBool(True)}) & = \texttt{BoolVal(True)} \\
\mathcal{ZB}(\texttt{BBool(False)}) & = \texttt{BoolVal(False)} \\
\mathcal{ZB}(\texttt{BNot}(b)) & = \texttt{Not}(\mathcal{ZB}(b)) \\
\mathcal{ZB}(b_1 \texttt{ and } b_2) & = \texttt{And}(\mathcal{ZB}(b_1), \mathcal{ZB}(b_2)) \\
\mathcal{ZB}(b_1 \texttt{ or } b_2) & = \texttt{Or}(\mathcal{ZB}(b_1), \mathcal{ZB}(b_2)) \\
\mathcal{ZB}(b_1 \texttt{ xor } b_2) & = \texttt{Xor}(\mathcal{ZB}(b_1), \mathcal{ZB}(b_2)) \\
\mathcal{ZB}(a_1 \otimes a_2) & = \mathcal{ZA}(a_1) \otimes \mathcal{ZA}(a_2) \qquad \text{for } \otimes \in \{==, <, <=, >, >=\} \\
\mathcal{ZB}(\texttt{BTruthy}(a)) & = (\mathcal{ZA}(a) \neq 0)
\end{array}
$$

3.3.3 Mapping of First-Order Logic formulas ($\mathcal{ZF}$)

Formulas such as Unary Operators (`!`), Binary Operators (`&&`, `||`, `=>`), and general BExp specifications are routed directly to Z3's logical connectives:

$$
\begin{array}{ll}
\mathcal{ZF}(\texttt{SpecFromBExp}(b)) & = \mathcal{ZB}(b) \\
\mathcal{ZF}(\texttt{SpecUnOp}(\texttt{!}, f)) & = \texttt{Not}(\mathcal{ZF}(f)) \\
\mathcal{ZF}(\texttt{SpecBinOp}(\texttt{\&\&}, f_1, f_2)) & = \texttt{And}(\mathcal{ZF}(f_1), \mathcal{ZF}(f_2)) \\
\mathcal{ZF}(\texttt{SpecBinOp}(\texttt{||}, f_1, f_2)) & = \texttt{Or}(\mathcal{ZF}(f_1), \mathcal{ZF}(f_2)) \\
\mathcal{ZF}(\texttt{SpecBinOp}(\texttt{=>}, f_1, f_2)) & = \texttt{Implies}(\mathcal{ZF}(f_1), \mathcal{ZF}(f_2))
\end{array}
$$

3.3.4 Quantifiers and Domains

The `SpecQuant` mapping dictates how $\forall$ (`Forall`) and $\exists$ (`Exists`) are bound over specific domains. For a bound variable x and formula body ϕ:

- **Infinite Domains (Z3 Native Quantifiers)**

  If the domain is unbounded (`DomainType`), `Z3` native quantifiers are utilized directly:

$$\begin{aligned} \mathcal{ZF}(\forall x \in \texttt{INT} \cdot \phi) &\implies \texttt{ForAll}(x_{z3}, \mathcal{ZF}(\phi)) \\ \mathcal{ZF}(\exists x \in \texttt{REAL} \cdot \phi) &\implies \texttt{Exists}(x_{z3}, \mathcal{ZF}(\phi)) \end{aligned}$$

- **Bounded Continuous/Discrete Domains**

  For `DomainRange` (discrete bounds $[lo, hi]$):

$$\begin{aligned} \mathcal{ZF}(\forall x \in [lo, hi] \cdot \phi) &\implies \texttt{ForAll}(x_{z3}, \texttt{Implies}(lo \leq x_{z3} \leq hi, \mathcal{ZF}(\phi))) \\ \mathcal{ZF}(\exists x \in [lo, hi] \cdot \phi) &\implies \texttt{Exists}(x_{z3}, \texttt{And}(lo \leq x_{z3} \leq hi, \mathcal{ZF}(\phi))) \end{aligned}$$

  For `DomainInterval` (real bounds with strict/non-strict operators), the bounds are dynamically translated to combinations of $<$ and $\leq$.

- **Finite Iteration Domains (Unrolled via finite connectives)**

  For lists or finite sets, the quantifier is unrolled during the mapping phase into large `And` or `Or` chains. Let $\mathcal{D}$ be a finite domain iterable returning values $\{v_0, v_1, \ldots, v_n\}$: (Note that the substitution is implemented with the help of `bound_var_map` dictionary.)

$$\begin{aligned} \mathcal{ZF}(\forall x \in \mathcal{D} \cdot \phi) &\implies \texttt{And}(\mathcal{ZF}(\phi[v_0/x]), \ldots, \mathcal{ZF}(\phi[v_n/x])) \\ \mathcal{ZF}(\exists x \in \mathcal{D} \cdot \phi) &\implies \texttt{Or}(\mathcal{ZF}(\phi[v_0/x]), \ldots, \mathcal{ZF}(\phi[v_n/x])) \end{aligned}$$

  This is applied to `DomainIdent` (array elements), `DomainValues` (literals), and `DomainVar` (variables in the lexical scope).

3.4 `VERI` instruction

Let $\mathcal{M}_{\texttt{Z3}}$ be the function outputting the corresponding solver when a configuration (a tuple of program state and scope) is given. Then, the abstract machine executing `SelVerIR`, has the following semantics:

$$\langle \texttt{VERI}\ \phi : c,\ e,\ \sigma,\ s,\ pc \rangle \quad \triangleright \quad \begin{cases} \langle c,\ e,\ \sigma,\ s,\ pc+1 \rangle & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi) \\ \langle c,\ e,\ \hat{\sigma}^{1},\ s,\ pc+1 \rangle & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \not\models {}^{2}\mathcal{ZF}(\phi) \end{cases}$$

[2]The abstract machine raises a `VerificationError` in that case.

[2]There may be several reasons for failure: ϕ may be false or undecidable. `Z3`-specific failures such as timeouts are also possible.

3.5 Soundness of FOL verification

The verification of FOL specifications in `SelVeri` is heavily dependent on `Z3`. Given their simplicity, the mappings described above are trivially sound, as their only purpose are putting the program state, scope and specifications into another form which is accepted by `Z3`. Assuming that `Z3` is sound [1, 2], then the whole FOL verification process (which is basically the composition of the mapping and `Z3` proving) is also sound.

4 Discrete time steps

To verify temporal logic formulae (both past and future LTL), the `SelVerifier` engine requires a well-defined notion of discrete time. Tracking every microscopic operation of the `SelVerIR` abstract machine (such as pushing and popping intermediate values from the stack) would lead to an explosion in the state space and is logically redundant. Instead, temporal progression must be synchronized with observable mutations in the program's memory, *id est*, scope and program state.

4.1 The `STEP` Instruction

To formalize this synchronization, the `SelVerIR` instruction set is augmented with a dedicated `STEP` command. During the compilation phase, the compiler strategically emits a `STEP` instruction immediately following any high-level statement that mutates the observable state or scope—specifically, after variable declarations (`DECL`, `LDECL`) and assignments (`STORE`, `LSTORE`).

The execution of the `STEP` instruction acts as a global clock tick for the verification engine, effectively partitioning the continuous execution into discrete, meaningful moments.

4.2 Operational Semantics and State Tracking

When the abstract machine encounters a `STEP` instruction, the verification engine's `on_step` hook is invoked. This momentarily halts standard execution to perform the following temporal maintenance routines in strictly this order:

1. **History Snapshot (for past-LTL):** The engine captures the current `RuntimeConfiguration` snapshot (comprising the current program state and scope) and appends it to the `history` array. It also initializes a new, empty dictionary in `pltl_memo` to cache temporal evaluations for this specific step.
2. **Global Clock Increment:** The internal monotonic clock (`last_step`) is incremented by exactly one.
3. **Automaton Advancement (for future-LTL):** The engine delegates the captured snapshot to `advance_future_obligations`. It iterates over all pending future obligations residing in the `fltl_pending` queue, evaluating their propositional atoms against the snapshot, and advancing their underlying Deterministic Finite Automata (DFA) to their respective next states.

By abstracting the granular execution steps into discrete memory snapshots, the `STEP` instruction elegantly bridges the gap between the imperative execution model of `SelVeri` and the strict discrete-time semantics of Linear Temporal Logic.

5 Verification of past-LTL formulae

Unlike static First-Order Logic (FOL) specifications, past-oriented Linear Temporal Logic (past-LTL or pLTL) formulae require reasoning over the execution history of the program. The `SelVerifier` engine evaluates these temporal specifications by maintaining a discrete trace of `RuntimeConfiguration` snapshots continuously generated during program execution.

5.1 Sequential Network Structure and Memoization

A naive recursive evaluation of past-LTL properties over an execution trace can lead to exponential time complexity. To circumvent this, `SelVerifier` implements a *sequential network structure* utilizing dynamic programming.

At any given execution step k, the verifier maps the past-LTL formula into a sequential evaluation network. The truth value of a temporal subformula at step k strictly depends on the truth value of its constituents at step k and, optionally, the memoized truth value of the same subformula at step $k-1$. This is implemented via the `pltl_memo` dictionary, mapping `(step, specification)` to its boolean outcome. This structural memoization unfolds the temporal logic evaluation into a linear sequential circuit over time, achieving $O(|H| \times |\phi|)$ complexity, where $|H|$ is the length of the execution history and $|\phi|$ is the size of the abstract syntax tree of the specification [3].

5.2 Execution Trace and Deductive Bounds

In standard past-LTL over infinite traces, properties evaluate back to time zero. However, in an imperative language like `SelVeri`, variables are dynamically introduced into the scope. To avoid verifying properties over states where the free variables of the specification do not yet exist, the verifier computes a strict lower bound, s_{init} (`start_step`), via the `pltl_deduce_inital_step` routine. This step represents the maximum of the declaration and initialization steps of all free variables present in the specification. The temporal evaluation strictly halts at s_{init}. It is also possible for user to specify s_{init} themselves, however this may lead to some runtime errors if the specified s_{init} is smaller then it can possibly be.

5.3 Semantics of Temporal Operators

Let $H = \langle \sigma_0, \sigma_1, \ldots, \sigma_k \rangle$ be the sequence of recorded states (history), where k is the current execution step. Let s_{init} be the initial valid step for the specification ϕ. We define the evaluation function $\mathcal{V}(\phi, k, s_{init})$ as follows:

- **Base Case (FOL):** If ϕ is a pure FOL formula, it is evaluated directly against the current state snapshot using `Z3`:

$$\mathcal{V}(\phi, k, s_{init}) = \mathcal{M}_{\texttt{Z3}}(\sigma_k, s_k) \models \mathcal{ZF}(\phi)$$

- **Previously ($\odot\phi$):** Evaluates to true if ϕ held in the immediately preceding state.

$$\mathcal{V}(\texttt{Previously}\ \phi, k, s_{init}) = \begin{cases} \text{False} & \text{if } k = s_{init} \\ \mathcal{V}(\phi, k-1, s_{init}) & \text{otherwise} \end{cases}$$

- **Once ($\Diamond_P\phi$):** Evaluates to true if ϕ held at any point in the valid past, including the present.

$$\mathcal{V}(\texttt{Once}\ \phi, k, s_{init}) = \mathcal{V}(\phi, k, s_{init}) \vee \Big(k > s_{init} \wedge \mathcal{V}(\texttt{Once}\ \phi, k-1, s_{init})\Big)$$

- **Historically ($\Box_P\phi$):** Evaluates to true if ϕ held at all points in the valid past, including the present.

$$\mathcal{V}(\texttt{Historically}\ \phi, k, s_{init}) = \mathcal{V}(\phi, k, s_{init}) \wedge \Big(k > s_{init} \implies \mathcal{V}(\texttt{Historically}\ \phi, k-1, s_{init})\Big)$$

- **Since ($\phi_1\ \mathcal{S}\ \phi_2$):** Evaluates to true if ϕ_2 held in the past, and from that moment onwards, ϕ_1 has strictly held.

$$\mathcal{V}(\phi_1\ \texttt{Since}\ \phi_2, k, s_{init}) = \mathcal{V}(\phi_2, k, s_{init}) \vee \Big(k > s_{init} \wedge \mathcal{V}(\phi_1, k, s_{init}) \wedge \mathcal{V}(\phi_1\ \texttt{Since}\ \phi_2, k-1, s_{init})\Big)$$

5.4 Soundness of past-LTL verification

The soundness of the past-LTL verification framework is established in three parts:

1. **Base Soundness:** The leaves of the temporal Abstract Syntax Tree (AST) are pure FOL formulae. As argued in **section 3**, the evaluation of these base FOL propositions using the `Z3Mapper` and the `Z3` solver is sound.

2. **Trace Finiteness:** The program emits a finite, discrete string of states H. The boundary condition s_{init} ensures that no temporal operator queries a state before its constituent variables have well-defined semantics.

3. **Structural Induction:** The recursive transition rules exactly mirror the well-established mathematical semantics of past-LTL over finite traces [4]. By construction, the sequential network computes the truth value of a formula of depth d at step k assuming the correct computation of formulae of depth $d-1$ at step k, and formulae of depth d at step $k-1$. By simultaneous structural induction on the temporal AST depth and mathematical induction on the discrete execution steps, the evaluation strictly preserves the semantics of standard finite-trace past-LTL.

6 Verification of future-LTL formulae

While past-LTL properties can be evaluated recursively over an existing history, future-oriented Linear Temporal Logic (future-LTL or fLTL) specifies conditions over execution states that have not yet occurred. Furthermore, because `SelVeri` programs inherently produce finite execution traces (from start to termination), standard infinite-trace LTL semantics are not directly applicable. Therefore, `SelVerifier` relies on the semantics of LTL over finite traces (LTL_f) [5].

6.1 Propositional Abstraction and DFA Generation

To verify fLTL specifications dynamically, `SelVerifier` employs an automata-theoretic approach. When an fLTL specification is encountered, the `FutureLTLMapper` abstracts the formula into a propositional LTL_f structure. It achieves this by extracting all pure First-Order Logic (FOL) leaf expressions and substituting them with unique propositional atoms (e.g., $p_0, p_1, \dots$) .

The resulting propositional LTL_f formula is converted into Negation Normal Form (NNF) . Using the `ltlf2dfa` pipeline, the engine constructs a corresponding Monadic Second-Order Logic (WS1S) program, which is then delegated to the `MONA` tool [6] . `MONA` computes a minimal Deterministic Finite Automaton (DFA) that accepts exactly the set of finite traces satisfying the LTL_f property.

6.2 Tracking Future Obligations

Once the DFA is constructed, the verifier instantiates a `FutureObligation` object, initialized to the automaton's start state, and registers it within the `fltl_pending` queue . This obligation continuously monitors the execution trace from its point of creation.

At each subsequent step of the program's execution:

1. **Atom Evaluation:** The verifier evaluates the truth values of the abstracted FOL atoms against the current program state (σ_k, s_k) using the `Z3Mapper` .
2. **Transition Resolution:** These boolean values are substituted into the DFA's transition guards using algebraic evaluation (`SymPy`) . Because the automaton is deterministic, exactly one valid transition is selected, updating the obligation's current state.
3. **Early Failure Detection:** To prevent unnecessary computation, the engine performs a reachability analysis on the DFA. If an obligation transitions into a state from which no accepting state can ever be reached (`can_reach_accepting`), a `VerificationError` is immediately raised .

6.3 Scope Semantics and Program Termination

Future obligations are inextricably linked to the lexical scope in which they are defined. If the program exits the scope that birthed a specific obligation, the variables referenced by its FOL atoms are destroyed. Consequently, the verifier intercepts scope-exit events and safely discards any pending obligations native to the dying scope. Although scope-exit is the natural step to end the tracking of the finite trace, `SelVeri` also allows users the specify the end-step they want; nevertheless, similar to pLTL, this usage is error-prone.

For the program execution to be deemed completely verified, all surviving future obligations must reach a designated accepting state upon program termination. During the `on_program_end` hook, the engine checks the final automaton state of every pending obligation. If any obligation remains in a non-accepting state, the fLTL property is violated, and the engine raises a failure .

6.4 Soundness Argument

The soundness of the future-LTL verification relies on the following pillars:

1. **Automaton Equivalence:** The translation from LTL_f formulas to DFAs via `MONA` has been extensively proven to be sound and complete [5, 6]. The generated DFA perfectly partitions satisfying and non-satisfying finite sequences of propositional assignments.

2. **Synchronous Evaluation:** The DFA transitions are perfectly synchronized with the discrete small-step operational semantics of the abstract machine. The state updates form an exact prefix of the trace sequence expected by the automaton.

3. **FOL Foundation:** The transition guards are resolved using `Z3`. Under the assumption that `Z3` is sound (as argued in **section 3**), the sequence of boolean inputs fed to the DFA exactly represents the true logical evaluation of the program's memory states.

7 Verification of separated-LTL formulae

A fundamental theoretical basis for combining past and future temporal operators is Gabbay's separation theorem [7, 8]. Formulated by Dov M. Gabbay, this theorem states that any arbitrary linear temporal logic formula containing both past and future operators can be rewritten into a logically equivalent separated form. In this form, specifications can be structured as boolean combinations of purely past, present, and future subformulae, often taking a "pLTL $\implies$ fLTL" shape where past conditions logically trigger future obligations. This separation is highly advantageous for executable temporal logic and system verification, as it avoids complex temporal overlaps and enables concise execution rules.

Separated Linear Temporal Logic (sLTL) specifications natively combine past and future temporal operators into a single, unified implication. In `SelVeri`, an sLTL formula acts as a conditional temporal guarantee: a future-LTL obligation is strictly enforced if and only if a specific past-LTL history condition holds at the current execution step.

7.1 Structure and Semantics

An sLTL specification is internally represented as a binary structure, comprising a past-LTL premise (the left-hand side, ϕ_p) and a future-LTL conclusion (the right-hand side, ϕ_f).

The operational semantics for verifying an sLTL formula at step k with a valid starting step s_{init} relies on the principle of vacuous truth:

1. **Initial Step Deduction:** If not explicitly provided, the engine deduces s_{init} for the past-LTL premise by determining the maximum initialization or declaration step among its free variables.
2. **Past Evaluation:** The verifier evaluates ϕ_p over the execution history up to the current step (k) using the `verify_past_LTL` routine.
3. **Future Delegation:**
   - If ϕ_p evaluates to `True`, the verifier triggers the future-LTL evaluation process, mapping ϕ_f into a state-tracking automaton and appending it to the pending future obligations.
   - If ϕ_p evaluates to `False`, the specification is vacuously `True` for the current state, and execution proceeds without tracking ϕ_f.

Formally, this evaluation step can be summarized as:

$$\mathcal{V}_{sLTL}(\phi_p \implies \phi_f, k) = \begin{cases} \text{Track } \mathcal{V}_{fLTL}(\phi_f, k) & \text{if } \mathcal{V}_{pLTL}(\phi_p, k, s_{init}) \text{ is True} \\ \texttt{true} & \text{otherwise} \end{cases}$$

7.2 Soundness Argument

The soundness of the sLTL verification seamlessly inherits the soundness guarantees of its underlying components. The evaluation reduces to a standard material implication $\phi_p \implies \phi_f$:

1. **Vacuous Truth:** If the past-LTL premise ϕ_p evaluates to false, the implication is vacuously true, which perfectly aligns with standard propositional logic semantics. The accurate evaluation of ϕ_p is guaranteed by the established soundness of the pLTL module.

2. **Delegated Soundness:** If ϕ_p evaluates to true, the truth value of the entire sLTL formula becomes strictly equivalent to the truth value of the future-LTL conclusion ϕ_f. Because the fLTL tracking via deterministic finite automata is shown to be sound, this delegated evaluation is also logically sound.

Consequently, because both the past history evaluation and the future state-tracking are sound, their conditional composition in `SelVeri` remains strictly sound.

References

[1] L. de Moura and N. Bjørner, "Proofs and refutations, and Z3," *Proceedings of the LPAR 2008 Workshops, Knowledge Exchange: Automated Provers and Proof Assistants, and the 7th International Workshop on the Implementation of Logics*, vol. 418. CEUR-WS.org, 2008.

[2] J. Doe and J. Smith, "Formal Assurance for Railway Interlockings: Verifying Z3 SAT Proofs with Tseitin in Rocq," *Proceedings of the 53rd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. Association for Computing Machinery, 2026.

[3] K. Havelund and G. Roşu, "Efficient monitoring of safety properties," *International Journal on Software Tools for Technology Transfer*, vol. 6, no. 2, pp. 158–173, 2004.

[4] O. Lichtenstein, A. Pnueli, and L. Zuck, "The glory of the past," *Logics of Programs*, Lecture Notes in Computer Science, vol. 193. Springer, Berlin, Heidelberg, pp. 196–218, 1985.

[5] G. De Giacomo and M. Y. Vardi, "Linear Temporal Logic and Linear Dynamic Logic on Finite Traces," *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 854–860, 2013.

[6] J. G. Henriksen, J. Jensen, M. Jørgensen, N. Klarlund, B. Paige, T. Rauhe, and A. Sandholm, "Mona: Monadic second-order logic in practice," *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, vol. 1019. Springer, Berlin, Heidelberg, pp. 89–110, 1995.

[7] D. M. Gabbay, "Declarative Past and Imperative Future: Executable Temporal Logic for Interactive Systems," *Proceedings of the Colloquium of Temporal Logic in Specification*, Lecture Notes in Computer Science, vol. 398. Springer, Berlin, Heidelberg, pp. 67–89, 1989.

[8] D. M. Gabbay, I. Hodkinson, and M. Reynolds, *Temporal Logic: Mathematical Foundations and Computational Aspects*, Volume I. Oxford University Press, 1994.